

Mini RFC: The QUIC Transport Protocol

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

QUIC is a secure general-purpose transport protocol. This document defines version 1 of QUIC, which conforms to the version-independent properties of QUIC.

QUIC is a connection-oriented protocol that creates a stateful interaction between a client and server.

The QUIC handshake combines negotiation of cryptographic and transport parameters. QUIC integrates the TLS handshake, although using a customized framing for protecting packets. The handshake is structured to permit the exchange of application data as soon as possible. This includes an option for clients to send data immediately (0-RTT), which requires some form of prior communication or configuration to enable.

We use **the green color** to denote optional features in this project.

Related optional features: *QUIC Handshake* (20pt), *0-RTT Transmission* (5pt)

Endpoints communicate in QUIC by exchanging QUIC packets. Most packets contain frames, which carry control information and application data between endpoints. QUIC authenticates the entirety of each packet and encrypts as much of each packet as is practical. QUIC packets are carried in UDP datagrams to better facilitate deployment in existing systems and networks.

Related optional features: *Packet Protection* (5pt)

Application protocols exchange information over a QUIC connection via streams, which are ordered sequences of bytes. Two types of streams can be created: bidirectional streams, which allow both endpoints to send data; and unidirectional streams, which allow a single endpoint to send data. A credit-based scheme is used to limit stream creation and to bound the amount of data that can be sent.

Related optional features: *Unidirectional Stream* (5pt), *Flow Control* (15pt)

QUIC provides the necessary feedback to implement reliable delivery and congestion control. QUIC depends on congestion control to avoid network congestion.

QUIC connections are not strictly bound to a single network path. Connection migration uses connection identifiers to allow connections to transfer to a new network path. Only clients are able to migrate in this version of QUIC. This design also allows connections to continue after changes in network topology or address mappings, such as might be caused by NAT rebinding.

Once established, multiple options are provided for connection termination. Applications can manage a graceful shutdown, endpoints can negotiate a timeout period, errors can cause immediate connection teardown, and a stateless mechanism provides for termination of connections after one endpoint has lost state.

Related optional features:
Connection ID (5pt), *Connection Migration* (20pt)

Related optional features: *Idle Timeout* (5pt), *Stateless Reset* (5pt)

2 Connection

A QUIC connection is shared state between a client and a server.

Each connection starts with a handshake phase, during which the two endpoints establish a shared secret using the cryptographic handshake protocol and negotiate the application protocol. The handshake confirms that both endpoints are willing to communicate and establishes parameters for the connection.

QUIC relies on a combined cryptographic and transport handshake to minimize connection establishment latency.

Figure 1 shows a simplified handshake and the exchange of packets and frames that are used to advance the handshake. Exchange of application data during the handshake is enabled where possible, shown with an asterisk (*). Once the handshake is complete, endpoints are able to exchange application data freely.

In Project I, we need to exchange almost nothing in the handshake so the handshake is simplified to be very simple, as shown in Figure 2. It is not a standard QUIC handshake yet. We will add the missing pieces, such as connection ID negotiation, cryptographic key generation, and transport parameters exchange, later when implementing the optional features.

For connections that are no longer needed or desired, there are several ways for a client and server to terminate a connection: idle timeout, immediate close, stateless reset.

An endpoint sends a CONNECTION_CLOSE frame to terminate the connection immediately. A CONNECTION_CLOSE frame causes all streams to immediately become closed; open streams can be assumed to be implicitly reset.

After sending a CONNECTION_CLOSE frame, an endpoint immediately enters the closing state. In the closing state, an endpoint retains only enough information to generate a packet containing a CONNECTION_CLOSE frame and to identify packets as belonging to the connection. An endpoint in the closing state sends a packet containing a CONNECTION_CLOSE frame in response

HTTPS handshake requires 3 RTT due to the separation of TCP and TLS handshake.

We use the blue color to mark sentences that are not cited from the RFCs.

We discuss only the immediate close way in this document. *Idle Timeout* (5 pt) and *Stateless Reset* (5pt) are optional features.

Graceful termination is tough. Think about why we need the four-way handshake termination in TCP.

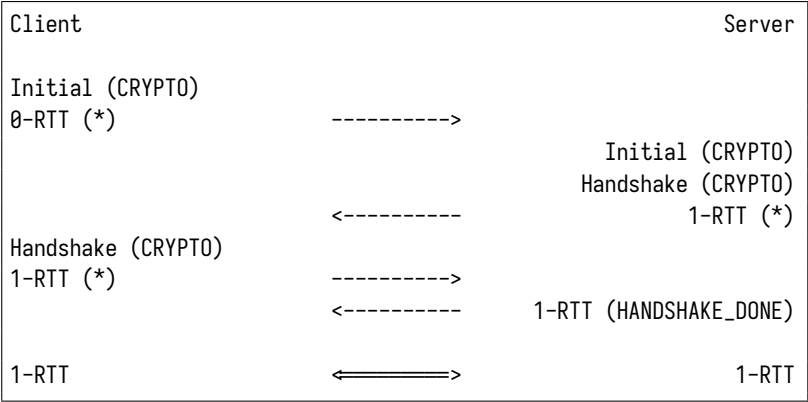
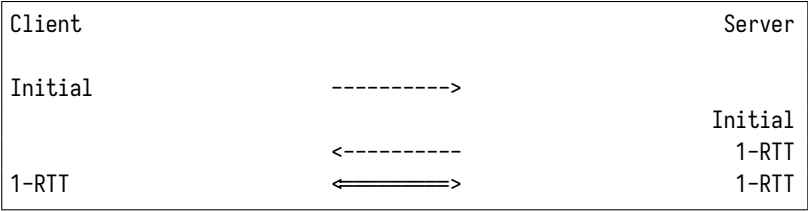


Figure 1: *Simplified QUIC Handshake*



We omit almost the entire handshake! It looks like just the three-way handshake of TCP now.

Figure 2: *Even More Simplified QUIC Handshake*

to any incoming packet that it attributes to the connection.

After receiving a `CONNECTION_CLOSE` frame, endpoints enter the draining state. While otherwise identical to the closing state, an endpoint in the draining state **MUST NOT** send any packets.

The closing and draining connection states exist to ensure that connections close cleanly and that delayed or reordered packets are properly discarded. These states **SHOULD** persist for at least **500ms**.

RFC specifies a value derived from RTT. We temporarily use a constant value until *Probe Timeout* (10 pt) is implemented.

3 Stream

Streams in QUIC provide a lightweight, ordered byte-stream abstraction to an application. Streams can be **unidirectional** or bidirectional.

Streams can be created by sending data. Other processes associated with stream management – ending, canceling, and managing flow control – are all designed to impose minimal overheads. For instance, a single `STREAM` frame can open, carry data for, and close a stream. Streams can also be long-lived and can last the entire duration of a connection.

Streams can be created by either endpoint, can concurrently send data interleaved with other streams, and can be canceled. QUIC does not provide

any means of ensuring ordering between bytes on different streams.

QUIC allows for an arbitrary number of streams to operate concurrently and for an arbitrary amount of data to be sent on any stream, **subject to flow control constraints and stream limits**.

Streams can be **unidirectional** or **bidirectional**. **Unidirectional streams carry data in one direction: from the initiator of the stream to its peer**. Bidirectional streams allow for data to be sent in both directions.

Streams are identified within a connection by a numeric value, referred to as the stream ID. A stream ID is a 62-bit integer (0 to $2^{62} - 1$) that is unique for all streams on a connection. Stream IDs are encoded as variable-length integers, see §4.1.2. A QUIC endpoint **MUST NOT** reuse a stream ID within a connection.

The least significant bit (0x01) of the stream ID identifies the initiator of the stream. Client-initiated streams have even-numbered stream IDs (with the bit set to 0), and server-initiated streams have odd-numbered stream IDs (with the bit set to 1).

The second least significant bit (0x02) of the stream ID distinguishes between bidirectional streams (with the bit set to 0) and unidirectional streams (with the bit set to 1).

The two least significant bits from a stream ID therefore identify a stream as one of four types, as summarized in Table 1.

Bits	Stream Type
0x00	Client-Initiated, Bidirectional
0x01	Server-Initiated, Bidirectional
0x02	Client-Initiated, Unidirectional
0x03	Server-Initiated, Unidirectional

Table 1: *Stream ID Types*

The stream space for each type begins at the minimum value (0x00 through 0x03, respectively); successive streams of each type are created with numerically increasing stream IDs. A stream ID that is used out of order results in all streams of that type with lower-numbered stream IDs also being opened.

STREAM frames encapsulate data sent by an application. An endpoint uses the Stream ID and Offset fields in STREAM frames to place data in order.

Endpoints **MUST** be able to deliver stream data to an application as an ordered byte stream. Delivering an ordered byte stream requires that an endpoint buffer any data that is received out of order, up to the advertised flow control limit.

An endpoint could receive data for a stream at the same stream offset multiple times. Data that has already been received can be discarded.

Streams are an ordered byte-stream abstraction with no other structure visible to QUIC. STREAM frame boundaries are not expected to be preserved when data is transmitted, retransmitted after packet loss, or delivered to the application at a receiver.

Optional Features: *Flow Control* (10 pt)

$2^{62} - 1$ is a magic number in QUIC.
 $2^{62} = 4 \text{ EB} = 4096 \text{ PB}$.

We use only client-initiated bidirectional streams in this project.

It's not a big issue since a STREAM frame will be sent when the stream is actually used later and the framework may allocate resource for the stream then. But these implicitly created streams should be counted in the stream limit if *Flow Control* (10 pt) is implemented.

4 Packetization

4.1 Encoding

4.1.1 Notational Conventions

Packet and frame diagrams in this document use a custom format. Complex fields are named and then followed by a list of fields surrounded by a pair of matching braces. Each field in this list is separated by commas.

You need to "parse" a QUIC packet to retrieve its content.

Individual fields include length information, plus indications about fixed value, optionality, or repetitions. Individual fields use the following notational conventions, with all lengths in bits:

- $x (A)$: Indicates that x is A bits long
- $x (i)$: Indicates that x holds an integer value using the variable-length encoding described in §4.1.2
- $x (A..B)$: Indicates that x can be any length from A to B ; A can be omitted to indicate a minimum of zero bits, and B can be omitted to indicate no set upper limit; values in this format always end on a byte boundary
- $x (L) = C$: Indicates that x has a fixed value of C ; the length of x is described by L , which can use any of the length forms above
- $x (L) = C..D$: Indicates that x has a value in the range from C to D , inclusive, with the length described by L , as above
- $[x (L)]$: Indicates that x is optional and has a length of L
- $x (L) \dots$: Indicates that x is repeated zero or more times and that each instance has a length of L

This document uses network byte order (that is, big endian) values. Fields are placed starting from the high-order bits of each byte.

By convention, individual fields reference a complex field by using the name of the complex field.

When a single-bit field is referenced in prose, the position of that field can be clarified by using the value of the byte that carries the field with the field's value set. For example, the value `0x80` could be used to refer to the single-bit field in the most significant bit of the byte, such as One-bit Field in Figure 3.

4.1.2 Variable-Length Integer Encoding

QUIC packets and frames commonly use a variable-length encoding for non-negative integer values. This encoding ensures that smaller integer values need fewer bytes to encode.

QUIC heavily employs variable-length encoding.

The QUIC variable-length integer encoding reserves the two most significant bits of the first byte to encode the base-2 logarithm of the integer encoding length in bytes. The integer value is encoded on the remaining bits, in network byte order.

```

Example Structure {
  One-bit Field (1),
  7-bit Field with Fixed Value (7) = 61,
  Field with Variable-Length Integer (i),
  Arbitrary-Length Field (..),
  Variable-Length Field (8..24),
  Field With Minimum Length (16..),
  Field With Maximum Length (..128),
  [Optional Field (64)],
  Repeated Field (8) ...,
}

```

Figure 3: *Example Format*

This means that integers are encoded on 1, 2, 4, or 8 bytes and can encode 6-, 14-, 30-, or 62-bit values, respectively.

2 Most Significant Bits	Length	Usable Bits	Range
00	1	6	0-63
01	2	14	0-16383
10	4	30	0-1073741823
11	8	62	0-4611686018427387903

Table 2: *Summary of Integer Encodings*

4.2 Packet

QUIC endpoints communicate by exchanging packets. Packets are carried in UDP datagrams. A UDP datagram can include one or more QUIC packets. The datagram size refers to the total UDP payload size of a single UDP datagram carrying QUIC packets. The datagram size includes one or more QUIC packet headers and payloads, but not the UDP or IP headers. The maximum datagram size is defined as the largest size of UDP payload that can be sent across a network path using a single UDP datagram. .

QUIC uses the long packet header during connection establishment. Packets with the long header are Initial, 0-RTT, Handshake, and Retry. Packets with the short header are designed for minimal overhead and are used after a connection is established.

4.2.1 Long Header

An Initial packet uses long headers with a type value of 0x00.

We set the maximum datagram size to 1472 (1500 [Ethernet MTU] - 20 [IP Header] - 8 [UDP Header]).

Maximum Transmission Unit Discovery is an optional feature that weights 5pt.

We don't implement the full handshake in Project I, so only Initial packets are required.

```

Initial Packet {
  Header Form (1) = 1,
  Fixed Bit (1) = 1,
  Long Packet Type (2) = 0,
  Reserved Bits (2),
  Packet Number Length (2),
  Version (32),
  Destination Connection ID Length (8),
  Destination Connection ID (0..160),
  Source Connection ID Length (8),
  Source Connection ID (0..160),
  Token Length (i),
  Token (..),
  Length (i),
  Packet Number (8..32),
  Packet Payload (8..),
}

```

Figure 4: *Initial Packet*

It contains these fields:

- **Header Form:** The most significant bit (0x80) of byte 0 (the first byte) is set to 1 for long headers.
- **Fixed Bit:** The next bit (0x40) of byte 0 is set to 1.
- **Long Packet Type:** The next two bits (those with a mask of 0x30) of byte 0 contain a packet type. 0x00 are assigned to Initial packets.
- **Type-Specific Bits:** The semantics of the lower four bits (those with a mask of 0x0f) of byte 0 are determined by the packet type.
- **Reserved Bits:** Two bits (those with a mask of 0x0c) of byte 0 are reserved. The value MUST be set to 0.
- **Packet Number Length:** The least significant two bits (those with a mask of 0x03) of byte 0 contain the length of the Packet Number field. [The value is set to 0 until we introduce the packet number in acknowledgment generation.](#)
- **Version:** The QUIC Version is a 32-bit field that follows the first byte. This field indicates the version of QUIC that is in use and determines how the rest of the protocol fields are interpreted.
- **Destination Connection ID Length / Destination Connection ID / Source Connection ID Length / Source Connection ID:** [Connection ID is not used in essential features. We set the lengths of both source and destination IDs as 0 to implicitly omit these fields.](#)

0x00000000 is reserved for version negotiation. 0x00000001 has been assigned to QUIC version 1.

Related optional features:
 Connection ID (5 pt), Connection Migration (15 pt)

- **Token Length / Token:** Token is not used in essential features. We set its length to 0 to implicitly omit these fields.
- **Length:** The length of the remainder of the packet (that is, the Packet Number and Payload fields) in bytes.
- **Packet Number:** Skip this field temporarily.
- **Payload:** This is the payload of the packet – containing a sequence of frames. It's empty since we skip the cryptographic handshake.

The Length field is used to decide the boundaries of packets in one UDP datagram, which is, however, not used in the basic part.

4.2.2 Short Header

QUIC defines a single packet type that uses the short packet header. A 1-RTT packet uses a short packet header. It is used after the connection is established.

```
1-RTT Packet {
  Header Form (1) = 0,
  Fixed Bit (1) = 1,
  Spin Bit (1),
  Reserved Bits (2),
  Key Phase (1),
  Packet Number Length (2),
  Destination Connection ID (0..160),
  Packet Number (8..32),
  Packet Payload (8..),
}
```

Figure 5: 1-RTT Packet

- **Header Form:** The most significant bit (0x80) of byte 0 is set to 0 for the short header.
- **Fixed Bit:** The next bit (0x40) of byte 0 is set to 1.
- **Spin Bit:** The third most significant bit (0x20) of byte 0 is the latency spin bit. Spin Bit is an optional feature of version 1 of QUIC, so we set it to 0.
- **Reserved Bits:** The next two bits (those with a mask of 0x18) of byte 0 are reserved. The value MUST be set to 0.
- **Key Phase:** The next bit (0x04) of byte 0 indicates the key phase, which allows a recipient of a packet to identify the packet protection keys that are used to protect the packet. We temporarily set it to 0.
- **Packet Number Length:** The least significant two bits (those with a mask of 0x03) of byte 0 contain the length of the Packet Number field. The value is set to 0 until we introduce the packet number in acknowledgment generation.

It should be set to a random number as specified in the standard. It's related to the observability of QUIC, which is out of the scope of this project.

Related Features: *Packet Protection* (5 pt)

- **Destination Connection ID:** Connection ID is not used in Project I. Skip this field since we have configured its length to 0.
- **Packet Number:** Skip this field temporarily.
- **Packet Payload:** This is the payload of the packet.

4.3 Frames

The payload of QUIC packets consists of a sequence of complete frames. Version Negotiation, Stateless Reset, and Retry packets do not contain frames.

The payload of a packet that contains frames **MUST** contain at least one frame, and **MAY** contain multiple frames and multiple frame types. Frames always fit within a single QUIC packet and cannot span multiple packets.

A sender can minimize per-packet bandwidth and computational costs by including as many frames as possible in each QUIC packet. A sender **MAY** wait for a short period of time to collect multiple frames before sending a packet that is not maximally packed, to avoid sending out large numbers of small packets. An implementation **MAY** use knowledge about application sending behavior or heuristics to determine whether and for how long to wait. This waiting period is an implementation decision, and an implementation should be careful to delay conservatively, since any delay is likely to increase application-visible latency.

Each frame begins with a Frame Type, indicating its type, followed by additional type-dependent fields:

```
Frame {
  Frame Type (i),
  Type-Dependent Fields (..),
}
```

Figure 6: Generic Frame Layout

QUIC uses 20 types of frames now and we will introduce them when they are used in further mini RFCs. In this document, we introduce only three types of them: PADDING, STREAM, and CONNECTION_CLOSE.

4.3.1 PADDING Frames

A PADDING frame (type=0x00) has no semantic value. PADDING frames can be used to increase the size of a packet. PADDING frames are formatted as below, which shows that PADDING frames have no content. That is, a PADDING frame consists of the single byte that identifies the frame as a PADDING frame.

4.3.2 STREAM Frames

STREAM frames implicitly create a stream and carry stream data.

In Project I, Initial packets also do not contain frames. Only 1-RTT packets do.

You are allowed to send one packet for every frame. Frame Coalescing is an optional feature weights 5pt.

Think about Nagle's algorithm and the TCP_NODELAY option of TCP you may have learned in the networking course.

In this project, your implementation should include as many currently available frames as possible but are not required to wait for more frames.

```
PADDING Frame {
  Type (i) = 0x00,
}
```

Figure 7: *PADDING Frame Format*

The Type field in the STREAM frame takes the form 0b00001XXX (or the set of values from 0x08 to 0x0f). The three low-order bits of the frame type determine the fields that are present in the frame:

The OFF bit (0x04) in the frame type is set to indicate that there is an Offset field present. When set to 1, the Offset field is present. When set to 0, the Offset field is absent and the Stream Data starts at an offset of 0 (that is, the frame contains the first bytes of the stream, or the end of a stream that includes no data). The LEN bit (0x02) in the frame type is set to indicate that there is a Length field present. If this bit is set to 0, the Length field is absent and the Stream Data field extends to the end of the packet. If this bit is set to 1, the Length field is present. The FIN bit (0x01) indicates that the frame marks the end of the stream. The final size of the stream is the sum of the offset and the length of this frame.

Multiple frames, possibly from different streams, may be packed in one packet.

```
STREAM Frame {
  Type (i) = 0x08..0x0f,
  Stream ID (i),
  [Offset (i)],
  [Length (i)],
  Stream Data (..),
}
```

Figure 8: *STREAM Frame Format*

STREAM frames contain the following fields:

- **Stream ID:** A variable-length integer indicating the stream ID of the stream.
- **Offset:** A variable-length integer specifying the byte offset in the stream for the data in this STREAM frame. This field is present when the OFF bit is set to 1. When the Offset field is absent, the offset is 0.
- **Length:** A variable-length integer specifying the length of the Stream Data field in this STREAM frame. This field is present when the LEN bit is set to 1. When the LEN bit is set to 0, the Stream Data field consumes all the remaining bytes in the packet.
- **Stream Data:** The bytes from the designated stream to be delivered.

Stream Data may be empty. The sending peer can send a STREAM frame with no data but with the FIN bit to close a stream.

When a Stream Data field has a length of 0, the offset in the STREAM frame is the offset of the next byte that would be sent.

The first byte in the stream has an offset of 0. The largest offset delivered on a stream, i.e., the sum of the offset and data length, cannot exceed $2^{62} - 1$.

Stream Data contains data in the range [Offset, Offset + Length). [Offset, Offset) is implicitly empty.

4.3.3 CONNECTION_CLOSE Frames

An endpoint sends a CONNECTION_CLOSE frame (type=0x1c or 0x1d) to notify its peer that the connection is being closed. The CONNECTION_CLOSE frame with a type of 0x1c is used to signal errors at only the QUIC layer, or the absence of errors (with the NO_ERROR code). The CONNECTION_CLOSE frame with a type of 0x1d is used to signal an error with the application that uses QUIC.

Support for CONNECTION_CLOSE frames with error codes is part of the optional feature *Error Handling* (10 pt).

If there are open streams that have not been explicitly closed, they are implicitly closed when the connection is closed.

```
CONNECTION_CLOSE Frame {
  Type (i) = 0x1c..0x1d,
  Error Code (i),
  [Frame Type (i)],
  Reason Phrase Length (i),
  Reason Phrase (..),
}
```

Figure 9: CONNECTION_CLOSE Frame Format

CONNECTION_CLOSE frames contain the following fields:

- **Error Code:** A variable-length integer that indicates the reason for closing this connection. A CONNECTION_CLOSE frame of type 0x1c uses predefined codes. A CONNECTION_CLOSE frame of type 0x1d uses codes defined by the application protocol.
- **Frame Type:** A variable-length integer encoding the type of frame that triggered the error. A value of 0 (equivalent to the mention of the PADDING frame) is used when the frame type is unknown. The application-specific variant of CONNECTION_CLOSE (type 0x1d) does not include this field.
- **Reason Phrase Length:** A variable-length integer specifying the length of the reason phrase in bytes. Because a CONNECTION_CLOSE frame cannot be split between packets, any limits on packet size will also limit the space available for a reason phrase.
- **Reason Phrase:** Additional diagnostic information for the closure. This can be zero length if the sender chooses not to give details beyond the Error Code value.

You are allowed to always use NO_ERROR (0x00) error code and blank reason phrase unless you implement *Error Handling* (10 pt).

5 Application Protocol

5.1 Connection

This document defines a set of functions for QUIC connections that application protocols can rely upon.

- When implementing the client role, an application protocol can:
 - open a connection, which begins the exchange API: `CreateConnection()`
- When implementing the server role, an application protocol can:
 - listen for incoming connections, which prepares for the exchange API:
`SetConnectionReadyCallback()`
- In either role, an application protocol can:
 - immediately close the connection API: `CloseConnection`
 - [get notified when the peer close the connection](#) API: `SetConnectionCloseCallback`
 - [get notified when a new stream is opened by the peer](#) API: `SetStreamReadyCallback`

5.2 Stream

This document defines a set of functions on streams that application protocols can rely upon.

- On the sending part of a stream, an application protocol can:
 - write data API: `CreateStream / SendData`
 - end the stream (clean termination), resulting in a STREAM frame with the FIN bit set API: `CloseStream`
- On the receiving part of a stream, an application protocol can:
 - read data API: `SetStreamDataReadyCallback`